DS 8001

Negative Feedback and Improvement Priorities

Professional review of course materials, exercises, and assignment guidance

Purpose: identify recurring friction points in the DS8001 materials and translate them into constructive, actionable improvements. The feedback below is synthesized from the provided course documents and is not presented as student survey data.

Prepared from the supplied DS8001 course materials

Executive Summary

The materials provide substantial technical coverage, but several features can create avoidable difficulty for students. The most important concerns are consistency, density, assignment precision, and the balance between conceptual explanation and implementation requirements.

Inconsistent administrative guidance

Course information says assignments and projects can be completed in groups of 2, while both assignment sheets allow groups of 1-3. This should be reconciled in one authoritative location.

High information density

Lecture decks cover many algorithms, proofs, recurrences, implementations, and numerical methods. Several slides are text- and formula-heavy, which can make rapid review difficult.

Assignment instructions are constraint-heavy

Assignments impose detailed restrictions on imports, filenames, function signatures, file types, Python version, and execution structure. These rules are important, but the volume raises the risk of losing marks for format rather than algorithmic understanding.

Examples and correction notes can be hard to parse

Some solution materials contain handwritten annotations or OCR-like text, while some handouts include small correction notes. A clean, consolidated solution set would improve reliability.

Greedy vs. exact-method distinctions require strong emphasis

The course correctly shows that greedy methods may fail on 0-1 knapsack and can be suboptimal for set covering/dominating-set problems. Students may benefit from a clearer decision guide for when a greedy rule is provably safe.

Detailed Negative Feedback

Area	Concern	Why It Matters	Recommended Improvement
Course administration	Group-size guidance is inconsistent across the course overview and assignment sheets.	Students may be uncertain which rule governs graded work.	Publish a single current policy in D2L and repeat the same wording in every assignment.
Lecture design	Some topics are presented with dense formulas, pseudocode, proofs, and complexity analysis on the same slide sequence.	Students need to separate the core algorithmic idea from the proof and implementation details.	Use a consistent three-part pattern: intuition, pseudocode, then complexity/proof.
Exercise solutions	Some solution files include handwritten or OCR-like notes that are difficult to read.	Ambiguous notation can make correct reasoning look inconsistent or unreliable.	Provide a clean typed solution edition and keep annotations as a separate optional version.
Assignments	Submission rules are extensive and highly specific.	A technically correct solution can still risk deductions for structural mistakes.	Add a one-page pre-submission checklist with filenames, imports, signatures, Python version, and main-gate requirements.
Algorithm selection	Many paradigms are covered quickly: brute force, divide-and-conquer,	Students may know individual algorithms but still struggle to choose the	Add a comparison matrix showing problem signals, recurrence patterns, guarantees, and common failure

	greedy, dynamic programming, optimization, and numerical methods.	appropriate paradigm on a new problem.	cases.
Complexity analysis	The course repeatedly moves between exact operation counts, recurrences, and asymptotic notation.	Students can confuse O_n style counts with Big-O classes or omit work inside recursive calls.	Include a standard checklist: input size, elementary operation, recurrence/count, solution, final Big-O.
Greedy methods	Greedy examples deliberately include cases where the method fails or is approximate.	Without a compact proof framework, students may memorize examples rather than understand when greedy is valid.	Emphasize exchange arguments, cut/cycle properties, and counterexample construction as reusable proof tools.
Dynamic programming	DP introduces definition of subproblems, recurrences, base cases, evaluation order, and backtracking.	Missing any one component can produce an incomplete answer even when the core recurrence is understood.	Use the five-step DP template as a required answer structure in exercises and assignments.
Numerical methods	Later material shifts from algorithm design into optimization, matrices, factorization, and numerical computation.	The conceptual transition is significant and may feel abrupt.	Add a bridge section connecting asymptotic algorithm analysis to the cost of matrix operations and Newton steps.

Priority Actions

1. Resolve policy inconsistencies immediately

Align group-size and submission guidance across the syllabus, D2L, and assignment PDFs.

2. Create a one-page assignment compliance checklist

Make structural requirements easy to verify before submission.

3. Standardize the presentation of algorithm analysis

For every major algorithm, use: problem -> idea -> pseudocode -> correctness -> complexity -> failure conditions.

4. Publish clean typed exercise solutions

Separate official solutions from handwritten annotations or informal notes.

5. Add a paradigm-selection study guide

Compare brute force, divide-and-conquer, greedy, dynamic programming, and numerical optimization using recognizable problem signals and examples.

Evidence from the Provided Materials

Course overview

The course overview lists assignments and the project as work that can be done individually or in groups of 2, and covers prerequisites spanning data structures, algorithms, greedy methods, dynamic programming, recursion, and complexity analysis.

Assignment 1

The assignment permits groups of 1-3 and includes detailed restrictions on imports, filenames, function signatures, main(), .py submission, and Python 3.11+.

Assignment 2

The second assignment also permits groups of 1-3 and repeats detailed submission constraints, while adding NumPy, SciPy, pandas, optimization, matrix computation, simulated annealing, enumeration, and dynamic programming.

Divide-and-conquer materials

The materials cover recurrence solving, recursive complexity, Towers of Hanoi, MergeSort, inversion counting, and maximum subarray with detailed proofs and analysis.

Greedy materials

The materials explicitly distinguish cases where greedy is optimal, approximate, or unsuccessful, including stable matching, dominating/set covering, coin change, interval scheduling, shortest paths, and MSTs.

Dynamic programming materials

The lecture presents a five-step DP design structure and covers weighted interval scheduling, segmented least squares, knapsack, Bellman-Ford, negative cycles, and sequence alignment.

Optimization and numerical methods

The later lecture covers convexity, steepest descent, Newton's method, matrix-operation costs, Gaussian elimination, substitution, LU, and Cholesky factorization.

Overall Assessment

The core content is rigorous and broad. The main negative feedback is therefore not that the course lacks substance, but that the volume and precision of the material can create unnecessary cognitive and administrative friction. The highest-value improvements are consistency, cleaner solution presentation, explicit checklists, and a stronger recurring framework for selecting and analyzing algorithms.

Source Documents Reviewed

- DS8001_lec00_courseInfo.pdf
- DS8001_f26_HW1.pdf
- DS8001_F26_HW2.pdf
- DS8001_lec01_introToAlgos_handout.pdf
- DS8001_lec01ex_introAlgos.pdf
- DS8001_lec01exSoln_introAlgos.pdf
- 8001_lec2_divideAndConquer_ex.pdf
- 8001_lec2_divideAndConquer_exSoln.pdf
- DS8001_lec2_divideAndConquer_handout.pdf
- 8001_lec03_greedyAlgos_ex.pdf
- 8001_lec03_greedyAlgos_exSoln.pdf

- 8001_lec03_greedyAlgos_handout.pdf
- 8001_lec04_DP_ex.pdf
- 8001_lec04_DP_exSoln.pdf
- 8001_lec04_DP_handout.pdf
- DS8001_lec05_unconstrainedOpt_ex.pdf
- DS8001_lec05_unconstrainedOpt_handout.pdf